
CSC3060 Tutorial 2

Mingyi BAO
2026 Week 4

In this tutorial

1. Visualization: The Binary Model
2. Modern C++ Behavior
3. Advanced Testing

Visualization: The Binary Model

We all know that computers store information in **binary**.

Example:

0: 0000 0000 0000 0000 0000 0000 0000 0000

1: 0000 0000 0000 0000 0000 0000 0000 0001

-1: 1111 1111 1111 1111 1111 1111 1111 1111

Visualization: Hex vs Bin

To make things clearer, we often use hex.

Example:

0: 0x00000000	0xA: 10 0000 ... 1010
1: 0x00000001	0x5: 5 0000 ... 0101
-1: 0xFFFFFFFF	0xC: 12 0000 ... 1100

Visualization: The Binary Model

Here is a 32-bit signed integer:

0x7FFFFFFF 2147483647

What if we add 1 to it?

0x80000000 -2147483648

Visualization: View in Hex/Bin

- We could write a helper function that prints a number in hex or binary for us.
- But why not let our tools do the job for us?

- GDB/LLDB hex: `p/x variable_name`
- GDB/LLDB bin: `p/t variable_name`
- VSCode: Add to WATCH, then
`variable_name,h variable_name,b`

```
a = -5  
b = -3
```

> Registers

WATCH

```
a,b = 1111111111111111111111111111011  
b,h = 0xffffffffd
```

Modern C++ Behavior - Precedence

Bitwise operators have lower precedence than most of other operators.

`if (x & 1 == 0)` will be evaluated as

`if (x & (1 == 0))`

Use parentheses for every single operation!

Modern C++ Behavior - Right Shift

- Logical Shift: Fills the vacated bits with 0
- Arithmetic Shift: Fills with the **sign bit**

In C++, the right shift on a signed negative integer is **Implementation Defined**. But almost all modern compilers use AS.

Modern C++ Behavior - Right Shift

Example: `0xFFFFFFFF >> 1`

- `uint32_t: 0x7FFFFFFF`
- `int32_t: 0xFFFFFFFF`

How to switch?

`static_cast<>()`

Modern C++ Behavior - Shift by 32

What is the result of

`1 << 32`

Expect: 0

Actual: GCC - 0, Clang - 1 (on ARMv8)

Both of them have a warning for that (static)

```
test.cpp:6:22: warning: shift count >= width of type [-Wshift-count-overflow]
```

```
6 | int32_t result = x << 32;
```

```
^  ~
```

Modern C++ Behavior - Conversion

If you use smaller types

`int8_t, int16_t, ...`

C++ will promote them to `int` before doing the bitwise operations. So always be explicit and use `static_cast`

Modern C++ Behavior - Overflow

In C++, **signed integer** overflow is UB.

(But well defined for unsigned integer)

You don't need to handle these overflows; just let the bits warp around.

Advanced Testing

How to be a good tester?

- Consider “Extremes”
- Power of 2 values
- Random value testing

These will be the key skill for “Mystery Tasks”.

Advanced Testing

```
1  TEST(DataLabTest, AddRandom) {
2      constexpr int TEST_SIZE = 1e6;
3
4      std::random_device rd;
5      std::mt19937 gen(rd());
6      std::uniform_int_distribution<int32_t> dist(INT_MIN, INT_MAX);
7
8      int32_t x;
9      int32_t y;
10     int32_t sum;
11     for (int32_t i = 0; i < TEST_SIZE; i++) {
12         x = dist(gen);
13         y = dist(gen);
14         sum = x + y;
15         EXPECT_EQ(data_lab::add(x, y), sum)
16             << "Failed on add(" << x << ", " << y << ")";
17     }
18 }
```

Hint

- Thinking in Gates.
- If you get stuck, look at a Truth Table.
- Solve for 1-bit first, then scale up.
- We are here to help you on OH and Piazza.